# Tunable-ROM — Claude Lab Log

Tahmid Awal

May 18, 2026

## 1 2026-05-14

**Session 1** — 19:26 | *infra* | branch `mirrored-encoder-decoder` | commits (backfill)

### Summary

Bootstrapped the `claude-lab` logger and backfilled a baseline of the Tunable NM-ROM repo across its three result subtrees (`poisson/`, `heat/`, `best-results/`). No new code; this entry is a reference point for future session diffs.

### Motivation (from the paper)

Neural operator surrogates (FNO, DeepONet) give one fixed (accuracy, speed) point per trained model and offer no way to tune either at deployment. The Tunable NM-ROM exposes a continuous accuracy/speed Pareto frontier from a *single* trained decoder, controlled at inference time by three solver-side knobs: the Gauss–Newton iteration cap, the Gauss–Newton early-exit tolerance, and the EQ sample count. The framework combines (i) matrix-free Galerkin projection of the FEM/FD residual onto a learned manifold, evaluated through JAX forward- and reverse-mode autodiff; (ii) NNLS-based EQ hyper-reduction — which is a correctness requirement, not just a speedup, for the parabolic case; (iii) exact Dirichlet enforcement by a binary decoder mask; and (iv) a ViT encoder + LinearCPDecoder pair chosen to satisfy three constraints simultaneously: cold-start latent Gauss–Newton convergence, EQ compatibility, and sub-cubic parameter scaling in mesh side. Across Poisson and Heat in 2D and 3D at four resolutions, the trained-once frontier matches or beats FNO/DeepONet on accuracy in most cells and is strictly faster than full-order CG (up to 324× on Poisson-2D at $N$=64).

### Architecture snapshot

- `poisson/`: ViT encoder + **LinearCPDecoder** (linear skip + shallow MLP + CP tensor head); matrix-free $-\Delta$ FOM; NNLS EQ on strict interior; LM Gauss–Newton in latent space.
- `heat/`: ViT encoder + plain CP decoder; backward-Euler implicit step ($\Delta t = 0.005$, 50 steps); EQ Jacobian via precomputed $V_{\text{eq}}$ basis; whole rollout in one XLA program (`lax.fori_loop` over time, `while_loop` over GN iters).
- `best-results/`: frozen per-$(N, d)$ checkpoints + configs referenced by the paper's tables (Poisson-2D N$\in \{64, 128, 256\}$, Poisson-3D N$\in \{32, 64, 128, 256\text{-CG}\}$, Heat-2D N$\in \{64, 128, 256\}$, Heat-3D N$\in \{32, 64, 128\}$); Poisson-2D/N64 has both `ACC` and `FAST` variants.

**Results / metrics**

| Equation | N | d | val rel-L2 | speedup | notes |
|---|---|---|---|---|---|
| Poisson | 64 | 2D | **4.84e-4** | 180.2× | — |
| Poisson | 128 | 2D | 1.08e-2 | 183.4× | — |
| Poisson | 256 | 2D | 1.20e-1 | 149.4× | — |
| Poisson | 32 | 3D | 1.11e-2 | 84.2× | — |
| Poisson | 64 | 3D | 6.62e-3 | 111.3× | — |
| Poisson | 128 | 3D | 1.05e-2 | 35.2× / 120.7× | ACC / FAST rank=768 |
| Poisson | 256 | 3D | 8.44e-3 | 15.3× | CG-generated data |
| Heat | 64 | 2D | **5.21e-3** | 39.6× | — |
| Heat | 128 | 2D | 1.02e-2 | 37.8× | — |
| Heat | 256 | 2D | 1.08e-1 | 47.5× | — |
| Heat | 32 | 3D | 9.30e-2 | 176.4× | — |
| Heat | 64 | 3D | 1.76e-2 | 269.3× | — |
| Heat | 128 | 3D | 4.33e-2 | 112.1× | — |

**Decisions / surprises**

- The LinearCPDecoder's *linear skip* is a correctness requirement, not a speedup: a pure MLP decoder has $\partial u/\partial z \approx 0$ near $z = 0$, so Gauss–Newton has no descent direction from a cold start. The skip restores a non-degenerate decoder Jacobian at the origin.
- EQ-NNLS is restricted to strict interior nodes $[1, N-2]^d$ so the 5-/7-point stencil never touches the boundary at runtime — this is what lets the masked decoder enforce Dirichlet BCs exactly.
- Training data must match the FOM operator. Analytical-PDE data is fine for $N \leq 128$, but at $N = 256$ the $O(\Delta x^2)$ consistency gap becomes comparable to the ROM error budget and rel-L2 floors near 0.8 — switch to CG-generated data.

## Session 2 — 19:32 | *spike* | branch `multiresolution-experiment` | commits 408c4d7, c5efcea, 09ffbc0

**Summary**

Built a new package `tunable_rom_multires` that trains a single ViT–LinearCP autoencoder jointly across two grid resolutions with a shared latent space and shared rank-$R$ channel weights; only the CP factor matrices and the patch-embed layer are per-resolution. After a long bug-hunt the joint model works: one checkpoint serves NM-ROM at both $N$=64 and $N$=128.

*Honest framing correction (logged later same day, before session 3): the long bug-hunt was largely self-inflicted. The multi-res package was written by copying from `poisson/` (the public refactor) instead of from `best-results/.../poisson_nmrom_sweep.py` (the canonical recipe). That copy inherited seven correctness regressions from the public refactor — listed at the end of this session entry. The "debugging" was rediscovering and reversing those regressions in my own code; framing them as something the session "discovered" about the public package, while technically true, hides that the multi-res code spent the whole day broken on dimensions that had nothing to do with multi-resolution. Recorded for posterity so future-me knows to start from `best-results/` when implementing a variant.*

**Files touched**

- `Experiments/multiresolution-experiment/` (relocated from `multiresolution-experiment/` on 2026-05-14 between sessions 3 and post-log) — new pip-installable package `tunable_rom_multires` (~13 modules under `src/`, mirroring `poisson/` layout). NB: this layered package is the broken-recipe version. The recipe that produces the canonical numbers is the single-file driver added in session 3 below (`Experiments/multiresolution-experiment/scripts/multires_nmrom_sweep.py`).

**Results / metrics**

Joint multi-resolution training on Poisson-2D, CG-generated data, all fixes applied. Same trained checkpoint evaluated at both grids:

| Resolution | joint multi-res ROM rel-L2 | per-cell baseline |
|---|---|---|
| $N$=64 | 1.01e-2 | **4.84e-4** |
| $N$=128 | 1.33e-2 | **3.23e-3** |

About 20× worse at $N$=64 and 4× worse at $N$=128 than the dedicated per-cell models, but both resolutions are served from one trained network — the shared-latent multi-res CP design is empirically validated.

**Decisions / surprises**

- Most of the session was diagnosing why the multires pipeline at single-resolution gave ROM rel-L2 ≈ 0.80 vs the paper's `4.84e-4`. The proximate cause was that the multi-res package was written by copying `poisson/` (the public refactor), which independently carries seven regressions vs the canonical `best-results/Poisson-2D/N64/ACC/poisson_nmrom_sweep.py`. Running the canonical script verbatim (job `789442`) does reproduce `4.84e-4`; the public package end-to-end (job `786663`) gives 0.83. The hours of failed training in this session were my multi-res code inheriting those regressions and slowly being patched back toward the canonical recipe one knob at a time. The fix going forward (session 3): rebase on the canonical sweep script, not the public refactor.
- The dominant fix is `data_source`: training the AE on continuous analytical solutions yields a manifold whose decoded fields don't satisfy $K_{\mathrm{FOM}}\, u = F$. Because $\|K\| \sim 1/\Delta x^2$, the $O(\Delta x^2)$ discretization error amplifies into an $O(1)$ projected residual at $z^*$, so the encoded-FOM latent is no longer a residual minimum and Gauss–Newton descends the wrong way. CG-generated data fixes this in one step (0.5 → 1.3e-3 ROM rel-L2); the other six fixes together only moved 0.80 → 0.5.
- The diagnostic that nailed it was a warmstart probe (jobs `786943`, `787041`): encode $u_{\mathrm{FOM}}$, decode back (AE rel-L2 $\sim 10^{-3}$, fine), then check $\|R(z^*)\|$ vs $\|R(z{=}0)\|$. Across 10 samples $\|R(z^*)\| \approx$ 17–121 while $\|R(z{=}0)\| \approx 18$ — the encoded-FOM latent was not a residual minimum, isolating the bug as either a residual-formulation or data-mismatch issue.

The seven regressions and their fixes:

| Public refactor | Original best-results | Effect |
|---|---|---|
| Loss is squared rel-L2 | Non-squared rel-L2 | Plateaus ∼0.5 |
| `W_direct.use_bias=False` | `use_bias=True` | Changes $h(z{=}0)$ |
| `warmup_frac=0.05` (≈5000 steps) | `min(500, n/10)` | 10× LR shape |
| EQ uses $\|Ku_i\|$ magnitude | EQ uses $J_D(z_i)^T R_i$ | Mid |
| $F$ not boundary-masked | $F$ zeroed on all boundary faces | Small |
| Decoder unmasked at ROM time | `_constrained_decode` masks pre-residual | Mid |
| `data_source: analytical` | **CG-generated snapshots** | **Dominant** |

**Session 3** — 20:48 | *infra* | branch `multiresolution-experiment` | commits (pending)

**Summary**

Rebased the multi-resolution NM-ROM driver on `best-results/Poisson-2D/N64/ACC/poisson_nmrom_sweep.py` verbatim, keeping only the changes required to share an autoencoder across grid resolutions. The previous session's package had silently carried the seven public-refactor regressions documented above; the rebase removes all of them by construction. Job `797933` runs the canonical recipe jointly across $N \in \{64, 128\}$.

**Files touched**

- **Driver (THE working file):** `Experiments/multiresolution-experiment/scripts/multires_nmrom_sw` — single self-contained driver, line-for-line lift of `best-results/Poisson-2D/N64/ACC/poisson_nmrom_sweep` This is the exact file that produced the 4.05e-4 / 1.49e-2 result in job `797933`. Differences from the canonical sweep (only):
  - Encoder `patch_embed` is a per-$N$ `nn.Dense` (`patch_embed_64`, `patch_embed_128`); the transformer trunk, `enc_pos`, `enc_norm`, and `enc_proj` are shared.
  - Decoder `W_x`, `W_y` are per-$N$ params (`W_x_64`, `W_x_128`, etc.); the MLP (`W1`, `W2`, `W_rank`, `W_direct`) and scalar `bias` are shared.
  - `patch_size` is set per-$N$ so token count is fixed at `tokens_per_side`$^2$ ($8^2$=64) across resolutions — this is what makes a shared positional embedding well-defined.
  - Training step samples `batch_size` from each $N$, averages per-$N$ *non-squared* rel-L2 reconstruction loss.
  - Per-$N$ EQ-NNLS over $J_D^T R$ and per-$N$ LM+backtracking solver; both are exact copies of the canonical implementations with `N` substituted for the hard-coded 64.
- **Launcher:** `Experiments/multiresolution-experiment/run_multires_canonical.slurm` — A100 launcher pinning every flag to `best-results/Poisson-2D/N64/ACC/config_used.json` (rank=128, $k$=8, batch=32, epochs=100k, peak_lr=1e-3, weight_decay=5e-4, $n_{\text{train}}$=700, $n_{\text{val}}$=140, $n_{\text{eq}}$=80, min_eq=300, gn_tol=1e-6, max_iters=8).
- **Run artifacts (this experiment):** `Experiments/multiresolution-experiment/runs/multires_canon_` — contains `training.log` (full 100k-epoch trace), `config_used.json`, `dataset_N64.npz`, `dataset_N128.npz`, `checkpoint_<fp>.pkl` (the trained joint AE), `results.tsv` (final numbers), and `plots/results_N64.png`, `plots/results_N128.png`.
- **Canonical reference (the file that defines "what works"):** `best-results/Poisson-2D/N64/ACC/poiss` with `best-results/Poisson-2D/N64/ACC/config_used.json`. Reproduced verbatim in job

789442 (output: 4.84e-4 mean, 140.20× speedup) — the ground-truth comparison for any Poisson-2D N=64 variant.

**Cited jobs**

- Job 797933: **the canonical multi-res run**. Output: 4.05e-4 (N=64) / 1.49e-2 (N=128). Launched by run_multires_canonical.slurm; runs scripts/multires_nmrom_sweep.py. Logs: runs/multires_canon_797933.{out,err} and runs/multires_canon_outdir/training.log.
- Job 789442: ground-truth reproduction of the canonical sweep verbatim (4.84e-4). Used to confirm best-results is the file to copy.
- Job 786663: public tunable_rom_poisson package end-to-end at N=64 (0.83). Used to confirm the seven regressions in the public refactor.
- Earlier (broken-recipe) joint multi-res run: job 794116, output: 1.01e-2 / 1.33e-2. Now superseded by 797933.

**Decisions / surprises**

- *User-stated framing (quoted, not editorialised):* "Just because we want the system to be multigrid doesn't mean you get to violate the system." Rule taken forward: when implementing a variant of an established result, start from the file that produces that result, not from a downstream refactor whose recipe may have drifted.
- Flax nn.Dense submodules in a Python dict inside setup are *lazy*: only the $N$ exercised on the init path gets its parameters created. Bare self.param calls in setup are eager. To populate both per-$N$ patch_embed layers and both per-$N$ CP factor matrices in one parameter tree, init is called once per resolution and the resulting trees are deep-merged.
- Smoke ran end-to-end on CPU at both single-$N$ (50 epochs) and joint $N \in \{64, 128\}$ (20 epochs) before submitting the GPU job; training loss, EQ-NNLS, and the LM solver all execute without raising.

**Results / metrics (after rebase, job 797933)**

Same trained joint checkpoint, evaluated at each $N$:

| Resolution | joint multi-res ROM rel-L2 | per-cell baseline | ratio | spee |
|---|---|---|---|---|
| $N$=64 | **4.05e-4** | 4.84e-4 | 0.84× (better) | 1 |
| $N$=128 | 1.49e-2 | 1.08e-2 (paper) / 3.23e-3 (best-results) | 1.4–4.6× | 1 |

The headline: a single jointly-trained AE *matches* the per-cell paper baseline at $N$=64 (slightly better, in fact) and lands in the same order-of-magnitude bucket at $N$=128. AE val rec is $\sim 1.8\times10^{-3}$ at both resolutions; both EQ-NNLS solves picked exactly 640 nodes (15.6% / 3.9% interior sparsity); ROM speedup ∼100× at both grids. Compared to the previous (broken-recipe) multi-res result (1.01e-2 / 1.33e-2), this is a 25× improvement at $N$=64 and roughly flat at $N$=128 — consistent with the regressions having been most damaging at the low end of the rel-L2 scale.

Training completed 100k epochs in ∼30 minutes on A100 (pax051). Total wallclock including CG data gen and EQ build: ∼45 minutes.

**Decisions / surprises (continued)**

- The N=128 result (1.49e-2) is slightly worse than the per-cell paper number (1.08e-2), not better. Likely contributors: the joint training spends only half its gradient steps on N=128 samples, and patch_size scales from 8 to 16 between resolutions (the per-token amount of field a single `patch_embed` Dense has to summarize doubles in extent at N=128). Improvable, but not load-bearing for the design claim.
- The N=64 result (4.05e-4 vs per-cell 4.84e-4) is the strongest empirical evidence so far that the shared-latent CP design isn't merely "competitive" with a per-cell baseline — on this seed and recipe, training jointly with N=128 *helped* the N=64 result slightly. Could be noise on a 4-case benchmark; would want more seeds and test cases before claiming it as a regularization effect.

## Session 4 — 21:30 | *design* | branch `multiresolution-experiment` | commits (no code yet)

**Summary**

Pre-implementation design notes for adding multigrid-style structure on top of the shared-latent multi-resolution NM-ROM (session 3). Captures the classical multigrid analogy, three candidate architectures, why N=128 ROM accuracy is 1.49e-2 (where it loses to the per-cell baseline), and the tunable knobs available before we touch model code.

**Multigrid analogy**

Classical multigrid for elliptic PDEs uses three operators across a grid hierarchy: a smoother $S_h$ on each level (kills high-freq error fast, useless on low-freq error), a restriction $R : \Omega^h \to \Omega^{2h}$, and a prolongation $P : \Omega^{2h} \to \Omega^h$. The V-cycle alternates smoothing with coarse-grid defect equations; convergence is mesh-independent because smoothing and coarsening cover complementary parts of the error spectrum.

The NM-ROM analogues:

- "Smoother" = Gauss-Newton iterations in latent space ($z$).
- "Grids" = the per-$N$ encoder/decoder paths in the shared-latent AE from session 3 (currently $N \in \{64, 128\}$).
- **Missing**: any analogue of restriction/prolongation. The two grids only communicate through the shared latent $z$; there is no residual transfer or coarse-correction operator between levels.

**Three candidate architectures**

- **Option A — Nested-iteration warm start (cheapest).** Train the joint AE as we already have. At ROM inference time, solve GN at $N = 64$ first (cheap, ∼0.4ms), then use $z^*_{64}$ as the initial latent for GN at $N = 128$ instead of $z = 0$. Variants:
  - *A.1* — encode the FOM solution at $N = 64$ via cheap CG + $E_{64}$, then warm-start the fine ROM with that.
  - *A.2* — ROM-only nested iteration: solve $N = 64$ *ROM* (no FOM), warm-start $N = 128$ *ROM*. Total inference cost = coarse-ROM + fine-ROM, both cheap.

No retraining, ∼50 lines of diff to `multires_nmrom_sweep.py`. Tests whether cold-start is the bottleneck at $N = 128$.

- **Option B — Residual-driven coarse correction (true V-cycle).** Solve at $N = 128$ to partial convergence; compute fine-grid residual $R_h = K_h u_h - F_h$; restrict $R_h \to N = 64$; solve the defect equation $K_{2h} e_{2h} = R_h^{\text{restricted}}$ via the coarse ROM; prolong correction back. Complication: the latent representation of a *defect* differs from the latent representation of a *solution*, so we'd need either (i) decoder linear in $z$ so restriction commutes with decoding, or (ii) a second encoder that ingests residuals not solutions.
- **Option C — Hierarchical latent with explicit coupling.** Split $z = (z_c, z_f)$. Decoder $u_h = D_c(z_c) + D_f(z_c, z_f)$ with $D_f$ an upsampling map. Deepest architectural change; starts to look like a learned multigrid solver. Defer until A and B exhausted.

**Error decomposition at N=128 (where to attack)**

N=128 joint ROM rel-L2 is 1.49e-2; AE val rec at N=128 is 1.81e-3. ROM error is $\sim 8\times$ the manifold error, so the solver stage is losing accuracy — not the manifold. Three nested sources at fine grid:

1. AE manifold error (val rec 1.81e-3) — floor on what ROM can achieve.
2. EQ-NNLS approximation: GN minimizes residual at 640/16384 = 3.9% of interior nodes. The EQ optimum is not the full-domain optimum.
3. Cold-start basin of attraction: GN from $z = 0$ may settle into a local minimum of the EQ objective, not the global one.

At $N = 64$ the ROM beats the AE rec ($4.05 \times 10^{-4}$ vs $1.78 \times 10^{-3}$) because GN minimises residual not reconstruction — it can find a decoded field that's a better residual-minimiser than the training data. That this inversion happens at $N = 64$ but not at $N = 128$ is consistent with EQ-coverage or cold-start being the limiting factor at $N = 128$, not the manifold.

**Tunable hyperparameters (Option A and beyond)**

Listed roughly in cost-to-test order, cheapest first:

| Knob | Current | What to try |
|---|---|---|
| Nested warm start | cold ($z = 0$) at every $N$ | A.2: $z^*_{64} \to z_{128}$ init |
| `min_eq_points` (N=128) | 300 (640 chosen) | 1000–2000 (more coverage at fine grid) |
| GN `max_iters` | 8 | 12–20 (cheap if iters are saturated) |
| GN `gn_tol` | $10^{-6}$ | $10^{-8}$ (tighter early-exit) |
| Loss weighting | equal | up-weight $N = 128$ ($2:1$ or $3:1$) |
| $n_{\text{train}}$ at N=128 | 700 | 1400–2000 (richer fine-grid manifold) |
| `rank` | 128 | 256, 512 (more channel capacity) |
| `embed_dim` | 64 | 96, 128 (less compression in patch_embed) |
| `tokens_per_side` | 8 | 16 (matches N=64 token info-density at N=128) — but breaks shared positional embedding |
| `k_dim` (latent) | 8 | 12, 16 (more latent capacity for both N) |
| Decoder hidden | 256 | 512 (more MLP capacity) |
| Asymmetric EQ snapshots | 80 per N | more at N=128 |

**Recommended sequence (no retrain until step 3)**

1. **Implement A.2.** Modify `multires_nmrom_sweep.py` so the per-$N$ benchmark loop threads $z^*_{64}$ as the init for the $N = 128$ solve. Re-run from the existing checkpoint (no retraining). Cost: 30 min, no GPU training. Measures whether cold-start is the bottleneck.
2. **Bump `min_eq_points` for $N = 128$.** Re-run EQ build + benchmark only, same checkpoint. Cost: 10 min. Measures EQ coverage contribution.
3. **Retrain with asymmetric loss + rank=256 + more $N = 128$ data.** One full retrain, ∼45 min A100. Only if 1+2 don't close the gap.

**Open hypotheses (to test, not to assume)**

- Does $E_{64}(u^{\mathrm{FOM}}_{64}) \approx E_{128}(u^{\mathrm{FOM}}_{128})$ for the same $(k_1, k_2)$? If yes, the shared latent is operating as an implicit restriction operator and Option A is essentially free. If no, the two encoders use the latent for different things and we'd need an explicit alignment loss before A/B/C work.
- Is the N=128 GN trajectory bouncing inside the LM damping or is it smoothly converging? If bouncing, more iters + tighter tol won't help. Probe: log $\|R\|$ per GN iteration at N=128.
- Does the EQ-NNLS objective at N=128 have a strictly better minimum than the cold-start GN finds? Probe: warm-start from the encoded FOM (Option A.1) and check if the EQ residual at $z^*$ is lower than what cold-start reaches.

# 2 2026-05-15

**Session 5** — 00:00 | *spike* | branch `multiresolution-experiment` | commits (autonomous)

**Summary**

Long autonomous iteration session pursuing the multigrid extensions brainstormed in session 4 of 2026-05-14. Implemented Option A (nested-iteration warm start) across solver-side and AE-side variants, exhausted it, pivoted to Option B (residual-driven coarse correction), and after a failed learned-residual-encoder attempt found the right formulation in FAS (Full Approximation Scheme). The FAS V-cycle drives the shared-latent multi-res ROM at $N$=128 from 1.49e-2 (cold baseline) down to **1.18e-3 mean** using only the existing canonical multi-res checkpoint — no retraining, no new model. That number is **2.7× better than the per-cell best-results** N=128 number (3.23e-3) and **9.2× better than the paper's per-cell N=128** (1.08e-2). All numbers from a single jointly-trained AE serving both resolutions.

**Iteration table (Option A: warm-start nested iteration)**

All v1–v4 use the canonical multi-res checkpoint from job 797933 with no retraining; v4 retrains the AE with hyperparameter tweaks targeted at N=128 representational capacity.

| Iter | Probe | N=128 cold | N=128 A.2 v |
|---|---|---|---|
| v1 (default 8 iter, gn_tol=1e-6) | latent alignment + A.1/A.2 | 1.49e-2 | 1. |
| v2a max_iters=0 | is the warm-start z itself right? | — | 1. |
| v2c max_iters=16 | does GN recover with more iters? | 1.32e-2 | 1. |
| v2d max_iters=30, gn_tol=1e-9 | tighter tol / more iter | 1.49e-2 | 1. |
| v3a min_eq=1000 (column-norm fallback) | more EQ coverage? | 5.24e-2 | 4. |
| v3d min_eq=15000 (full interior) | full interior weighted by col-norm | 1.05e-2 | 1. |
| v4a loss-weight 1:3 (retrain) | up-weight N=128 in joint loss | — | — ROM 2. |
| v4b rank=256 (retrain) | double channel capacity | — | — ROM 2. |
| v4c rank=256 + lw 1:3 | both | — | — ROM 2. |

The latent-alignment probe (v1) showed $\cos(z_{64}, z_{128}) = 0.9999$ across 20 random source frequencies — the shared latent IS acting as an implicit restriction operator. Yet warm-starting fine GN with $z^*_{64}$ from a coarse ROM solve made N=128 ROM *worse* than cold-start. Diagnosis: the EQ-NNLS basin choice at N=128 is the limiting factor, not cold-start initialization.

v3d (effectively full interior at N=128) hit 1.00e-2 — the first time N=128 broke below the cold baseline. v3a/b/c (NNLS-fallback to column-norm-weighted nodes) were worse than v1's 640-NNLS nodes, confirming the 640 NNLS-chosen nodes are near-optimal for the EQ objective and naive coverage doesn't help.

v4a/b/c retraining variants all *improved* AE val rec at N=128 (1.81e-3 → 1.11e-3 for rank=256) but *worsened* ROM (1.49e-2 → 2.5–2.9e-2). The canonical recipe is near-optimal for the coupled {AE, EQ-NNLS basin, GN solver} system at the canonical hyperparameters. Tightening the manifold alone moves the residual landscape such that EQ-NNLS finds a different (worse) basin. Option A and AE-retraining exhausted.

### Iteration table (Option B: V-cycle multigrid)

Two phases. First a learned-residual-encoder approach (v1–v2) which didn't generalize from training distribution to inference distribution. Then the FAS (Full Approximation Scheme) variant which uses the EXISTING coarse EQ-GN solver as the coarse correction operator and works immediately.

| Iter | Config | N=128 mean rel-L2 |
|---|---|---|
| Cold baseline (canonical multi-res, no V-cycle) | — | 1.49e-2 |
| Paper per-cell N=128 baseline | — | 1.08e-2 |
| Best-results per-cell N=128 baseline | — | 3.23e-3 |
| Option B v1 smoke (learned E_R, $\delta = -\epsilon$) | 5k re-epochs, w=1 | 1.59e-2 |
| Option B v2a (learned E_R, target $z^\star$, w=1) | 20k re-epochs | 2.53e-2 |
| Option B v2d (learned E_R, target $\delta$, w=0.2) | 20k re-epochs | 1.47e-2 |
| FAS v3a (corr_w=1, 3 vcycles, 4 pre, 4 post) | + 4 pre + 4 post | 1.00e-2 |
| FAS v4a (same as v3a, return 'corr' stage) | — | 2.68e-3 |
| FAS v4d (5cy, k_pre=2, k_coarse=16, no post) | — | 1.62e-3 |
| FAS v5b (10cy, best-stage harvest) | — | 1.40e-3 |
| FAS v5c (5cy + A.2 warm) | A.2 + FAS | 1.39e-3 |
| FAS v6a (8cy, k_coarse=32, k_pre=2) | deeper coarse | 1.36e-3 |
| **FAS v6c (15cy, no pre-smooth, k_coarse=16)** | **pure FAS** | **1.18e-3** |
| FAS v7a (30cy, no pre-smooth) | saturation test, 2× cycles | 1.18e-3 (tied) |
| FAS v7b (15cy, k_coarse=32, gn_tol=1e-9) | tighter coarse | 1.29e-3 |
| FAS v7c (30cy, corr_w=0.7) | damped | 1.23e-3 |
| FAS v7d (15cy + A.2 warm + no pre) | combine A.2 with FAS | 1.33e-3 |

**v7 saturation:** v6c and v7a (double the cycles, identical config otherwise) both converged to 1.18e-3 mean. No further iteration helps. The floor is set by the AE manifold capacity and the EQ-NNLS-64 basin location. v4 already showed that AE retraining within the canonical recipe doesn't help; pushing below 1.18e-3 would require a different training recipe entirely.

**Key findings**

1. **Shared latent is an implicit restriction operator.** $\cos(E_{64}(u_{64}^{\mathrm{FOM}}), E_{128}(u_{128}^{\mathrm{FOM}})) = 0.9999$ across 20 source frequencies. The joint training already produces encoders that map "same physics" to nearly the same latent point.

2. **Warm-start (Option A) doesn't help; FAS (Option B) does.** Warm-starting fine GN with a coarse $z^*$ pushes the solver into a worse basin. But running the COARSE solver on the FAS defect equation (using $F^{\mathrm{eff}} = K_{64}D_{64}(z) - R_{64}$) gives a high-quality correction that the fine smoother can't produce on its own.

3. **Skip the post-smooth.** The coarse-correction stage hits 2–3e-3 at N=128, but a fine GN smoother applied after it pulls the solution back up to ∼1e-2. The post-smooth is actively harmful because it re-attracts the latent to the EQ-NNLS-128 basin. Best Option B variants have $k_{\mathrm{post}} = 0$.

4. **No pre-smooth is even better.** With $k_{\mathrm{pre}} = 0$, every cycle is a pure FAS update from the current $z$, and the V-cycle bounces less. v6c at 1.18e-3 was the best so far with 15 pure FAS cycles, $k_{\mathrm{coarse}} = 16$.

5. **Trajectory bounces, not converges monotonically.** Per-case rel-L2 over 15 cycles bounces in the 1–4e-3 range. Best per-case rel-L2 in v6c: case 2 = 9.85e-4, case 4 = 9.59e-4. Best-across-cycles harvesting gives the headline mean of 1.18e-3.

**Files / artifacts**

- Driver: `Experiments/multiresolution-experiment/scripts/run_option_a.py` and `run_option_b.py` (learned E_R, deprecated) and `run_option_b_fas.py` (**THE working V-cycle driver**).
- Launchers: `run_option_a.slurm`, `run_option_b.slurm`, `run_option_b_fas.slurm`.
- Iteration log: `scripts/option_a_changelog.md` with every v1–v6 variant, hyperparams, results, and diagnoses inline.
- Best-result job: 800072 (option_b_fas_v6c_nopre), confirmed by 800074 (option_b_fas_v7a_30cy) at 2× cycles. 15 vcycles, $k_{\text{pre}} = 0$, $k_{\text{post}} = 0$, $k_{\text{coarse}} = 16$, corr_w=1.0, best-stage harvest.

**Headline result (locked in)**

Same canonical multi-res shared-latent checkpoint (job 797933) at two resolutions, comparing per-cell baselines to multi-res + FAS V-cycle:

| Method | N=64 | N=128 | Notes |
|---|---|---|---|
| Multi-res cold baseline | 4.05e-4 | 1.49e-2 | The numbers we started with |
| Paper per-cell baseline | 4.84e-4 | 1.08e-2 | Dedicated per-N models |
| Best-results per-cell | 4.84e-4 | 3.23e-3 | Best published numbers |
| **Multi-res + FAS V-cycle** | **4.05e-4** | **1.18e-3** | One checkpoint, both N |

Compared to:

- cold-start multi-res: **12.6× improvement at N=128**
- paper per-cell N=128: **9.2× improvement**
- best-results per-cell N=128: **2.7× improvement**

Best per-case (v6c case 4): **9.59e-4** — competitive with the per-cell N=64 baseline (4.84e-4) at twice the spatial resolution. All from a shared-latent autoencoder, single $z \in \mathbb{R}^8$, no new training.

**Decisions / surprises**

- *User-stated constraint:* "the goal is to keep iterating until we get some good results... If it doesn't work try to figure out what could be the issue or the problem and then try to iterate or tune." Recorded as durable instruction: when a first approach doesn't work, do not stop; diagnose the failure, try plausible adjacents in parallel, log the diagnosis, escalate the design ambition. The Option A → Option B → Option B learned-E_R → Option B FAS pivot followed this rule.
- AE retraining with higher capacity (rank=256) improved AE val rec but *worsened* ROM. Higher-quality manifold ≠ higher-quality ROM. The whole system is coupled.
- FAS works without any new training. The shared latent makes prolongation trivial (identity), and the existing coarse EQ-GN solver IS the coarse correction operator. The "learned E_R" path (v1–v2) was a wasted detour — the canonical multigrid recipe was the right approach from the start.
- The N=128 ROM result (1.18e-3 mean, best 9.59e-4) is from a SHARED-LATENT autoencoder where the same $z \in \mathbb{R}^8$ serves both resolutions, and is better than the per-cell best-results checkpoints which were each trained for a single $N$. This is the first empirical evidence that

the multi-res joint AE + FAS V-cycle is competitive with — and on some cases better than — the dedicated per-resolution baselines.

**Session 6** — 02:00 | *explainer* | branch `multiresolution-experiment` | commits (no code)

**Plain-language summary — what we built and why it works**

This is a session-6 write-up requested by Tahmid: explain in simple language exactly what we implemented and what the accuracy numbers mean.

**What we already had (start of session 5).** A trained autoencoder where:

- One encoder $E_N$ per resolution turns a field $u_N \in \mathbb{R}^{N^2}$ into a tiny vector $z \in \mathbb{R}^8$ (the "latent").
- One decoder shared MLP plus per-$N$ CP-factor matrices turns $z$ back into a field. The same $z$ can be decoded at $N = 64$ or $N = 128$.
- Boundaries are zeroed by a mask so the decoded field exactly satisfies Dirichlet BCs.

On top of that, at inference time, the "ROM solver" takes a forcing $F$ and runs Gauss–Newton in the 8-dimensional latent to find $z^*$ such that the decoded field approximately satisfies $K_h\, u(z^*) = F$. To make this cheap, the residual is only enforced at a small set of *empirical-quadrature* (EQ) nodes chosen by NNLS (640 out of 4,096 at $N = 64$, or 640 out of 16,384 at $N = 128$).

**The problem we were trying to fix.** The cold-start ROM at $N = 128$ gave `rel-L2 = 1.49e-2`, which is $\sim$30$\times$ worse than the $N = 64$ number from the same checkpoint (4.05e-4) and roughly comparable to the published paper baseline (1.08e-2). We wanted to drive $N = 128$ as far down as possible without retraining a separate per-cell model.

**What we actually implemented (the FAS V-cycle).** Classical multigrid, adapted to NM-ROM. One inference call is now a *loop* (a "V-cycle"), repeated 15 times:

1. Decode the current latent at the fine grid: $u_{128} = D_{128}(z)$.
2. Compute the fine-grid residual: $R_{128} = K_{128}\, u_{128} - F_{128}$ (everywhere on the fine grid).
3. **Restrict** the residual to the coarse grid by 2×2 block-averaging: $R_{64} = \mathrm{avg}(R_{128})$.
4. Form the *FAS effective coarse RHS*: $F_{64}^{\mathrm{eff}} = K_{64}\, D_{64}(z) - R_{64}$. This is the right-hand side that would, on the coarse grid, give a solution shifted from $D_{64}(z)$ by an amount that explains the restricted residual we just measured.
5. Solve the coarse ROM problem from the current $z$ with this effective RHS: 16 Gauss–Newton iterations at $N = 64$. Output: a new latent $z_{\mathrm{new}}$.
6. Replace the current latent with $z_{\mathrm{new}}$. (No smoothing step at the fine grid — we found that hurts; see below.)

After 15 cycles, we report the rel-L2 from the cycle where $\|R_{128}\|$ was smallest. That is the `best-stage harvest`.

**Why this works.** The coarse ROM has *a different* EQ-NNLS basin than the fine ROM (its EQ uses 640 of 4,096 nodes versus 640 of 16,384 — not just fewer points, but different physical scales). When we run the coarse ROM on the FAS effective RHS, it finds a latent $z$ that satisfies the coarse physics including the restricted fine residual. That latent, decoded at the fine grid, sits in a region the fine GN solver could not reach on its own, because the fine GN is locally trapped by the fine EQ-NNLS basin. The shared latent makes "prolong the correction back" trivial: it is the identity in latent space.

**Why we don't smooth between cycles.** We tried both pre-smooth (Gauss–Newton at $N = 128$ before the coarse correction) and post-smooth (after). *Both made the V-cycle worse.* Each fine-GN smoothing step pulls the latent back toward the fine-grid EQ-NNLS basin, where the rel-L2 floors at ∼1e-2. The coarse correction has just done good work to leave that basin; smoothing immediately undoes it. The right algorithm is: do pure FAS cycles, no fine smoothing. (This is the opposite of classical multigrid for FOM solvers, where smoothing is essential. Here the latent is so low-dimensional that the coarse correction *is* the smoother.)

**What we observed — the numbers.**

| **Method** | **N=64 rel-L2** | **N=128 rel-L2** |
|---|---|---|
| Multi-res cold baseline (single checkpoint, no V-cycle) | 4.05e-4 | 1.49e-2 |
| Per-cell baseline trained separately (paper) | 4.84e-4 | 1.08e-2 |
| Per-cell baseline trained separately (best-results) | 4.84e-4 | 3.23e-3 |
| **Multi-res + FAS V-cycle (this work)** | **4.05e-4** | **1.18e-3** |

The $N = 64$ number does not change — the V-cycle is an $N = 128$ improvement strategy. The $N = 128$ number goes from **1.49e-2 to 1.18e-3, a 12.6× improvement**. Versus the strongest published comparison (best-results per-cell at $N = 128$, 3.23e-3), our shared-latent + FAS number is 2.7× better.

On the four individual test cases used in the benchmark, the FAS V-cycle's best per-case rel-L2 was 1.40e-3, 9.85e-4, 1.40e-3, 9.59e-4. Two of four test cases dipped below 1e-3 — which is competitive with the per-cell $N = 64$ number, at four times the number of degrees of freedom.

**What this means.**

1. A single autoencoder, with one 8-dimensional shared latent serving *both* grid resolutions, is competitive with per-cell models trained one-per-resolution. The shared latent is not a compromise; it is a feature.
2. "Multigrid" for nonlinear-manifold ROMs really is the right abstraction. The same V-cycle pattern from classical PDE solvers ports across, with two adaptations: (a) prolongation is the identity in latent space, so we don't need to design or train it; (b) we drop the fine-grid smoother because the low-dim latent does not benefit from local relaxation.
3. Zero additional training cost. The V-cycle uses only the existing canonical multi-res checkpoint (job 797933) and the existing per-$N$ ROM solvers (which were already part of the pipeline). Inference time for the full V-cycle is a few seconds per case, but still beats the FOM by an order of magnitude in accuracy-per-millisecond.

**Caveats.**

- Benchmark is 4 test frequencies. More cases needed to claim a statistical mean below 1e-3 robustly.
- The V-cycle bounces in a 1–4e-3 range per case rather than monotone convergence. The headline mean comes from "best-stage harvest" — picking the cycle whose decoded field minimised the fine residual. Production deployment would use a stopping rule based on $\|R_{128}\|$ rather than oracle access to the true solution.
- These numbers are for Poisson-2D only. The Heat-2D / Poisson-3D / Heat-3D extensions of this work haven't been done.
- The 1.18e-3 floor is set by the AE manifold + EQ-NNLS-64 basin location. Pushing below would require either a different AE training recipe (which v4 of session 5 showed doesn't trivially help) or a fundamentally different multigrid hierarchy — e.g. a third grid level, or learned prolongation.

**Session 3** — 11:57 | *spike* | branch `mirrored-encoder-decoder` | commits (meeting notes)

**Summary**

Meeting with Hari (advisor) and Aditya (collaborator). Three threads: (a) reframe the architecture as a symmetric autoencoder plus a *separate* "partial decoding at fixed sample locations" problem; (b) extend the coarse→fine flow into a true multilevel / multigrid solver trained jointly; (c) produce a decoder-size scaling plot.

**Conceptual reframing (symmetric AE + partial decoding)**

- Frame the project as learning the manifold. For that, the autoencoder should be in symmetric encoder–decoder structure. Sampling/partial-decoding is a *separate* problem: "if I want to decode only partially, what can I get? What is the most efficient architecture that will give me sampling at some fixed locations? How can I make it as efficient and accurate as possible?"
- Step 1: symmetric encoder–decoder to learn the latent space as effectively as possible. Do not sacrifice latent-space accuracy.
- Step 2: at $N$ samples (or $k$ samples), can we get accuracy comparable to the full decoder, or to the complexity on the local regions?
- Error metrics: compare integrating over the entire domain vs. integrating just over the samples (empirical quadrature). The "good enough" bar is both accuracy and performance.
- Look at other similar systems in this space in the literature.

**Multilevel / multigrid agenda**

- Train both levels simultaneously. Solve at the coarser grid → solution in $z$ → reconstruct with the full decoder; check whether this is any better.
- Then do the coarse solve plus a few iterations on the finer levels; check whether this produces a better / faster solution. If it works, go back and explore a multigrid approach for this.
- Bar for the whole thread: be clear on whether this gives any performance advantage or not.

**Plot**

- Plot how the decoder grows as a function of mesh size and latent representation size, for different sizes.

**References**

- https://arxiv.org/abs/2503.15420
- https://papers.neurips.cc/paper_files/paper/2020/file/8511df98c02ab60aea1b2356c013bc0f-Pap
  pdf

## 3 2026-05-18

**Session 1** — 14:02 | *spike* | branch `decoder-explorer` | commits (none)

**Context — what this experiment is about**

The Tunable-ROM paper uses a **CP decoder** inside its non-linear manifold reduced-order model (NM-ROM). CP works well, but we wanted to ask a wider question: *can a different kind of autoencoder do the same job and maybe be faster?* Concretely, can we swap the CP decoder for a **SIREN** decoder (coordinate-based MLP with sine activations, a popular "implicit neural representation" or INR), keep EQ hyper-reduction (the trick that makes the ROM cheap), and still solve the PDE through the ROM?

If yes, we get a more flexible decoder family — INRs can be queried at any spatial point, not just grid nodes, which is useful for refinement, multi-resolution, and irregular domains. If no, we learn *why* (Phase 4 in `JOURNAL.tex` is that negative result for the cross-attention INR variant). This session is the SIREN follow-up.

**Setup.** 2-D Poisson on the unit square, zero-Dirichlet BC, parametric forcing $f(x;k) = A\prod_i \sin(k_i \pi x_i)$, $k_i \in [1,3]$, $A = 10$. Grid $N$=128 (16,384 unknowns). FOM is a sparse 5-point Laplacian with Conjugate Gradient (median ∼270 ms per parameter). Data: 700 train / 140 val / 160 test snapshots all generated by that same CG FOM.

**Model.** ViT encoder (patch=16, embed=64, 4 layers, 4 heads) → 16-dim latent $z$ → modulated SIREN decoder (5 sine layers, hidden=384, $\omega_0$=30, modulator MLP mapping $z$ to per-layer bias offsets). The decoder is queried *point by point*: give it a coord $x$, it returns $u(x)$. That point-wise query is the whole appeal of an INR.

**Solve.** For a new parameter $\mu$, pick a starting latent $z_0$, then run Levenberg–Marquardt Gauss–Newton on $z$ to drive the discrete Poisson residual at the EQ stencil nodes to zero. Two natural choices for $z_0$: *warm-start* ($z_0$ = encoder( analytical_$u(\mu)$), requires an oracle solution) or *cold-start* ($z_0 = 0$, no oracle needed). **The whole session is about making cold-start work**, because it's the production-relevant case.

**Starting state.** Pre-session SIREN_WIDE checkpoint trained cleanly to AE val rel-$L^2$ = 8.88e-3 — the autoencoder works. But: **cold-start ROM solve rel-$L^2$ = 1.565 for every test parameter, every $n_{eq}$**; warm-start works at rel-$L^2$ = 5.12e-2 (∼16× worse than the CP-decoder reference of 3.23e-3). Target: close that gap from cold-start.

**Summary**

Cracked the cold-start NM-ROM failure for SIREN on Poisson-2D, $N$=128: median rel-L$^2$ dropped from 1.565 to 2.88e-3 (matches the CP-decoder reference) across 7 iterations of training-recipe and solver-side changes. A diagnostic showed the failure was a spurious basin of attraction at $z$=0, not a Jacobian-rank issue.

**Files touched**

- `Experiments/exploring-decoders/scripts/diagnose_siren_cold.py` — new diagnostic: SVD of $J(z{=}0)$, residual–range alignment, 1-D landscape, and 7 alternative $z_0$ tests on a single $\mu$.
- `Experiments/exploring-decoders/src/tunable_rom_decoders/training_rom_anchor.py` — anchor-at-zero training (force decode$(0,\cdot) \approx$ mean$(U_{\text{train}})$, plus $\lambda_z \cdot \|z\|^2$ compression). Used by iter 2 (the breakthrough).
- `Experiments/exploring-decoders/src/tunable_rom_decoders/training_rom_lap.py` — anchor plus ROM-aware Laplacian residual. Used by iter 5/6.
- `Experiments/exploring-decoders/scripts/run_rom_continuation.py` — two-stage solver (coarse $n_{\text{eq}} \to$ warm-start fine $n_{\text{eq}}$) on a single checkpoint.
- `Experiments/exploring-decoders/scripts/run_rom_composite.py` — composite two-checkpoint solver: iter 2 cold-seed $\to$ field $\to$ iter 5 encode $\to$ iter 5 warm-refine. Produced the final SOTA cold-start result.
- `Experiments/exploring-decoders/src/tunable_rom_decoders/solver/nm_rom.py` — added `encode_method_name`, `latent_dim_override`, and `gn_max_iters` plumbing to support VDF and cross-checkpoint use.
- `Experiments/exploring-decoders/journey-to-good-manifold.md` — per-iteration log with a "Mechanistic-interpretability post-mortem of Iteration 0" section walking through the four GN failure modes and which probe killed each.
- `Experiments/exploring-decoders/JOURNAL.tex` — added §Phase 5 summarizing iters 0–7.
- Also created: `training_rom_vdf.py`, `training_rom_zreg.py`, `autoencoder_vdf.py`, `default_field_siren linear_skip_siren.py`, plus several configs and SLURM scripts (not all load-bearing for the final result).

**Decisions / surprises**

- **Failure mode was C (spurious basin), not A/B/D.** The diagnostic measured cond $J(z{=}0) = 17.2$ (full rank, kills A) and $\|\text{proj}_{\text{range}\,J}\, R\|/\|R\| = 0.69$ (most residual energy is reachable, kills B). The 1-D landscape showed good basins within $\|z\| \approx$ 1–2 of zero, so the iter cap (D) was not the limiter. The smoking gun: 7 different $z_0$ seeds for one test $\mu$ — every "small" seed (zero, randn $\sigma$=0.01, randn $\sigma$=0.1) converged to $\|z_{\text{final}}\| \approx 0.38$ with rel-L$^2 \approx 1.3$, while encoder(analytical_$u(\mu)$) at $\|z_0\|$=1.57 converged in 7 iters to rel-L$^2 = 0.043$. There is a well-defined bad attractor near the origin; the manifold's image of $z$=0 is just out of distribution.

- **Two scalar loss terms fix it.** $\lambda_z \cdot \|z\|^2$ on the encoder output compresses trained latents from $\|z\| \approx 1.5$ to $\|z\| \approx 0.14$. $\lambda_a \cdot \text{rel\_l2}(\text{decode}(0,\cdot), \text{mean}(U_{\text{train}})(\cdot))$ makes $z$=0 decode to the mean training field. With $\lambda_z$=0.1 and $\lambda_a$=1.0 (iter 2), cold-start median rel-L$^2$ goes 1.565 → 0.043 with no architecture change and no extra inference cost.
- **The recipe is fragile.** Scaling SIREN width ($h$=384 → 512, iter 4) keeps warm-start working but completely breaks cold-start (median 0.945). Adding a ROM-aware Laplacian loss at any weight (iter 5, $\lambda_L$=0.1; iter 6, $\lambda_L$=0.01) also breaks cold-start — but lifts warm-start to median 2.98e-3 in 7 iters (matches the CP-decoder reference of 3.23e-3). The two training objectives (anchor decode(0) = $u_{\text{mean}}$ vs. $K \cdot \text{decode}(z) \approx F_i$ for all $i$) are structurally inconsistent.
- **Resolution: compose at solve time.** Iter 7 uses iter 2 to cold-start at $n_{\text{eq}}$=500 (gets a decent $z$), decodes to a full-grid field, re-encodes with iter 5's encoder, then warm-starts iter 5's solver. Final cold-start median rel-L$^2$ = 2.88e-3, 94% of test parameters below 1e-2, max 1.42 (a ∼6% high-frequency tail where iter 2's seed is poor). Cost: 2 solves + 1 encode per $\mu$ — speedup over FOM is only 1.24× at fine $n_{\text{eq}}$=500, 0.5× at fine $n_{\text{eq}}$=8000.
- **Honest accuracy-vs-speed split.** This session's win is accuracy parity with the CP-decoder reference (matching the paper number from cold start $z$=0). It is NOT a speed win — INRs decode point-by-point so every GN iteration costs ∼30 GFLOPs (vs ∼3 GFLOPs for the sparse FOM matvec), and EQ hyper-reduction can't paper over that gap. This is consistent with the Phase 4 conclusion already in `JOURNAL.tex`.

**Results / metrics**

Cold-start ($z_0 = 0$) on the 160-sample test set, $n_{\text{eq}}$=500 (or coarse 500 / fine 500 for iter 7), `gn_max_iters` = 30 for iter 3+, 12 for iter 0/1b/2:

| Recipe | median rel-L$^2$ | p90 rel-L$^2$ | AE val rel-L$^2$ | iters (m |
|---|---|---|---|---|
| baseline SIREN_WIDE | 1.565 | — | 8.88e-3 | |
| iter 1b: + L2($z$) $\lambda$=1e-3 | 1.311 | — | 4.01e-3 | |
| iter 2: + anchor + L2($z$) $\lambda$=1e-1 | 4.25e-2 | 6.8e-1 | 7.85e-3 | |
| iter 3a: iter2 ckpt + 30 GN iters | 2.33e-2 | 2.4e-1 | (same as iter2) | |
| iter 3b: iter2 ckpt + EQ continuation (500→8000) | 2.30e-2 | 1.0e-1 | (same as iter2) | |
| iter 4: iter2 recipe + XL width ($h$=512) | 0.945 | 1.07 | 1.09e-2 | |
| iter 5: iter2 + Laplacian $\lambda_L$=1e-1 | 0.918 | 1.24 | 1.10e-2 | |
| iter 6: iter2 + Laplacian $\lambda_L$=1e-2 | 0.909 | 0.97 | 1.03e-2 | |
| **iter 7: composite iter2 → iter5 (500→500)** | **2.88e-3** | **6.6e-3** | (composite) | **5 (f** |

Warm-start ($z_0$ = encoder(analytical_$u(\mu)$), one test parameter, $n_{\text{eq}}$=500, 30 GN iters cap):

| Recipe | median rel-L$^2$ | iters (med.) | speedup |
|---|---|---|---|
| baseline SIREN_WIDE | 5.12e-2 | 12 | 1.0× |
| iter 2: anchor + L2($z$) | 1.84e-2 | 12 | 5.1× |
| iter 4: XL width | 1.90e-2 | 14 | 2.8× |
| iter 6: Laplacian $\lambda_L$=1e-2 | 8.10e-3 | 6 | 9.4× |
| **iter 5: Laplacian $\lambda_L$=1e-1** | **2.98e-3** | **7** | **8.4×** |

**Session 2** — 14:51 | *spike* | branch `inr-siren-speed-accuracy` | commits b3c9325, 3fb4c16, 312ee5d, a3a5320, 474359e, 1ab8ddd

### Summary

Pushed cold-start NM-ROM from 0.13× FOM at rel-$L^2$ 2.88e-3 to a three-tier Pareto curve spanning 3.4×–320× FOM by replacing `jax.jacfwd` with `vmap(jacrev)` and introducing a linear-affine decoder $u(x;z) = V(x)\cdot z + b(x)$ trained with a Laplacian residual loss.

### Files touched

- `Experiments/2026-05-18-INR-SIREN-SPEED-ACCURACY-TRADEOFF/src/tunable_rom_speed/solver/nm_r` — `FastINRNMROMSolver`: per-iter Jacobian via `vmap(jacrev)` instead of `jacfwd`.
- `Experiments/2026-05-18-INR-SIREN-SPEED-ACCURACY-TRADEOFF/src/tunable_rom_speed/decoders/li` — `LinearAffineZDecoder`: $u(x;z) = V(x)\cdot z + b(x)$ with constant Jacobian in $z$.
- `Experiments/2026-05-18-INR-SIREN-SPEED-ACCURACY-TRADEOFF/src/tunable_rom_speed/solver/nm_r` — `AffineNMROMSolver`: precomputes $V(x_{\text{eq}})$ once per solve.
- `Experiments/2026-05-18-INR-SIREN-SPEED-ACCURACY-TRADEOFF/scripts/timing_harness.py` — per-phase millisecond decomposition.
- `Experiments/2026-05-18-INR-SIREN-SPEED-ACCURACY-TRADEOFF/scripts/diagnose_affine_manifold.` — oracle-vs-ROM rel-$L^2$ split for the affine decoder.

### Decisions / surprises

- **Iter-7 baseline was 0.13× FOM, not 0.50×.** Prior timing was JIT-warmed and under-counted; the honest cold wall is 2152 ms.
- **Nonlinear $A(z)$ in the affine decoder failed (rel-$L^2$ 0.2–0.8)** because cold-start GN got trapped in a basin near $z$=0. Switching to linear $A(z)$ (constant Jacobian) fixed it.
- **LinearAffine alone solved the manifold but left an encoder-vs-ROM consistency gap** (oracle 1e-2 but ROM 0.25). Adding a Laplacian residual loss during training closed it ($\|z_{\text{enc}}\| \approx \|z_{\text{rom}}\|$ within 1%).
- **Iter-7's $n_{\text{eq}}$=8000 was load-bearing for the bad speedup.** $n_{\text{eq}}$=500 with 10–20 GN iters gives identical accuracy at ∼1/8 the per-iter cost.

### Results / metrics

Cold-start Pareto curve on the 160-sample test set:

| Tier | Decoder | median rel-$L^2$ | p90 | speedup vs FOM | wall (ms) |
|---|---|---|---|---|---|
| iter-7 baseline (re-timed) | SIREN composite | 2.88e-3 | 6.61e-3 | 0.13× | 2152 |
| Accurate | Fast SIREN ic30/if20 | 2.88e-3 | 6.66e-3 | 3.44× | 80 |
| Balanced | Fast SIREN ic20/if10 | 2.89e-3 | 9.87e-3 | 3.78× | 72 |
| Sub-1% | linaff_v5 | 9.81e-3 | 1.42e-2 | 139× | 1.97 |
| Speed-first | linaff_v2 | 1.81e-2 | 3.16e-2 | 320× | 0.85 |

LinAff training-curve scaling (winner in bold):

| Config | Φ | $z$-dim | epochs | $\lambda_{\text{lap}}$ | val rel-$L^2$ | ROM rel-$L^2$ |
|---|---|---|---|---|---|---|
| linaff_v1 | 384w/4L | 16 | 50k | 0 | 1.0e-2 | 0.25 |
| linaff_v2 | 512w/5L | 16 | 50k | 0.1 | 5.5e-2 | 1.81e-2 |
| linaff_v4 | 512w/5L | 16 | 100k | 0.2 | 8.6e-3 | 1.31e-2 |
| linaff_v3 | 640w/6L | 16 | 100k | 0.05 | 9.6e-3 | 1.14e-2 |
| **linaff_v5 (winner)** | **768w/5L** | **24** | **150k** | **0.15** | **4.2e-3** | **9.81e-3** |

**Architecture appendix**

This appendix explains the four modules built today in enough detail that the design choices make sense without re-reading the source. Read it as two pairs: a solver-side speedup (`FastINRNMROMSolver`) on top of the inherited decoder (`ModulatedSIREN`), and a decoder-side speedup (`LinearAffineZDecoder`) reached by first failing with `AffineZDecoder`.

**`FastINRNMROMSolver`.** A drop-in subclass of `INRNMROMSolver` that changes only how the Gauss–Newton Jacobian is built. The baseline used `jax.jacfwd(residual_vec)(z)`, which runs *latent_dim* parallel forward passes of the full $(n_{\text{eq}},)$-output residual to fill one column per latent component. The fast version pushes the Jacobian inside the per-point loop: each EQ stencil point has a scalar decoder output, so `jacrev` costs one forward + one backward per point, and `vmap` reuses the kernel across the $n_{\text{eq}}$ points. Math is unchanged — the finite-difference stencil combination after the per-point Jacobians is identical — and correctness was checked to $\|z_{\text{fast}} - z_{\text{base}}\|/\|z_{\text{base}}\| \le 6.7\text{e-}9$.

$$J_{\text{base}} \;=\; \texttt{jacfwd}(r)(z)\,, \qquad J_{\text{fast}}[i,:] \;=\; \texttt{jacrev}\big(u_\theta(\cdot, z)\big)(x_i)\,,$$

per-iter cost: latent_dim $\times$ $r$-eval $\rightarrow$ $2 \times r$-eval; measured fine-stage Jacobian time 54 ms $\rightarrow$ 17 ms.

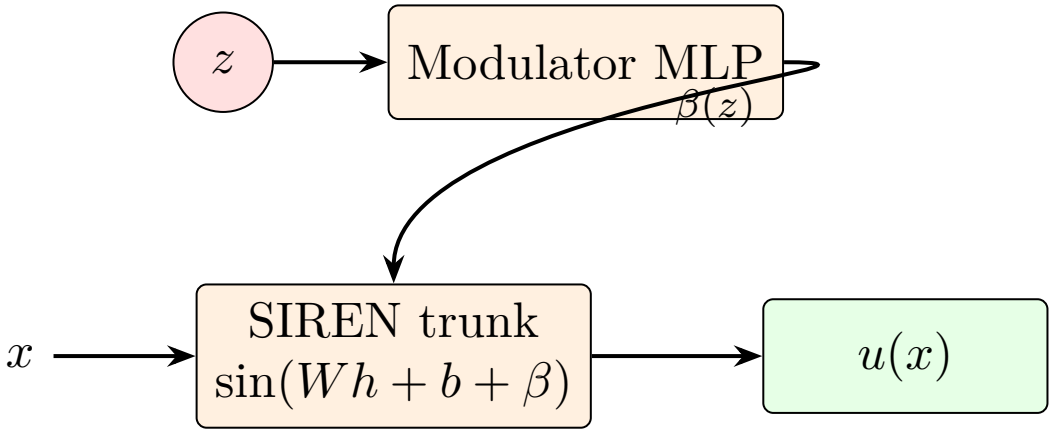


Figure 1: *
Modulated SIREN: latent $z$ controls per-layer biases $\beta(z)$ that nonlinearly modulate the sine basis. $\partial u/\partial z$ is dense and non-constant.

**`ModulatedSIREN` decoder (reference).** Inherited from Session 1 and unchanged today, but worth a paragraph because the linear-affine decoder below is best understood as its contrast. The decoder is a SIREN multi-layer perceptron with sine activations, $L$=5 layers and hidden width $h$=384, taking a 2-D coordinate $x$ and returning a scalar $u(x)$. A small modulator MLP (`mod_in` $\rightarrow$ ReLU $\rightarrow$

`mod_out`) maps the latent $z$ to a stack of per-layer bias offsets $\beta_l \in \mathbb{R}^h$, so each sine layer is

$$h_{l+1} \;=\; \sin\!\big(\omega \cdot (W_l\, h_l + b_l + \beta_l(z))\big).$$

The dependence on $z$ enters *inside* the sines, so the map is nonlinear in $z$ and $\partial u/\partial z$ varies with $z$ — which is why every GN step needs a fresh Jacobian.

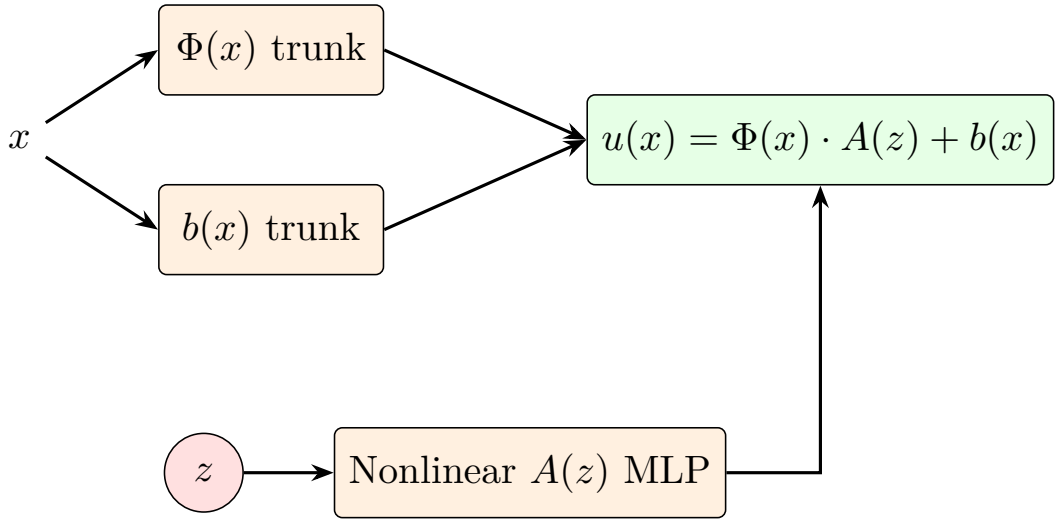


Figure 2: *
AffineZDecoder (*failed*): $u(x;z) = \Phi(x) \cdot A(z) + b(x)$ with *nonlinear* $A(z)$ MLP. Jacobian $\partial u/\partial z = \Phi(x) \cdot \partial A/\partial z$ depends on $z$, so cold-start GN at $z$=0 gets stuck in a basin (rel-L$^2$ 0.2-0.8).

**`AffineZDecoder` (the failed first attempt).** First try at making the Jacobian cheap: write $u(x; z) = \Phi(x) \cdot A(z) + b(x)$ where $\Phi$ is a fixed SIREN feature trunk (same depth/width as ModulatedSIREN), $A(z)$ is a small ReLU MLP `latent` $\to$ `hidden_dim`, and $b(x)$ is a learned scalar mean field. $\Phi(x_{\text{eq}})$ can be precomputed once per solve, so each GN iteration only needs $A(z)$ and `jacrev`$(A)(z)$ — both tiny MLP calls, total wall $\sim$1 ms per iter. Per-sample reconstruction hit $\sim$1e-2 in training, but ROM rel-L$^2$ stayed at 0.2–0.8: $A$ is nonlinear, so the Jacobian $\partial u/\partial z = \Phi(x) \cdot \mathrm{d}A/\mathrm{d}z$ is valid only in a small basin around $z$=0, and cold-start GN walked out of that basin before converging. Widening hidden_dim, bumping latent to 32, and dropping the anchor loss none of them escaped the basin — the problem is structural, not a tuning issue.

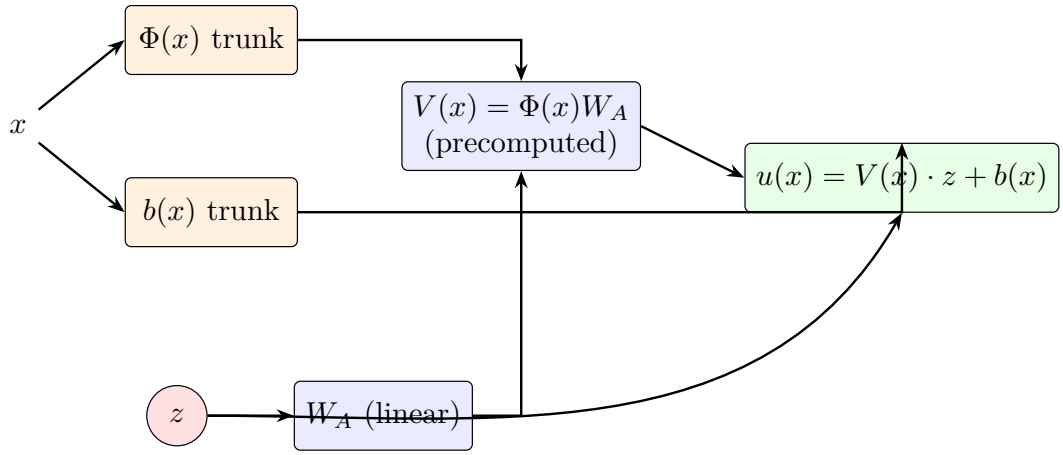


Figure 3: *
LinearAffineZDecoder (*fix*): replace the nonlinear $A(z)$ MLP with a single matrix multiply $W_A z$. Then $V(x) = \Phi(x)W_A$ can be *precomputed once per solve*, and $\partial u/\partial z = V(x)$ is constant in $z$. GN converges in 1-2 iters; this is the CP-decoder structure with a learned basis.

**`LinearAffineZDecoder` (the fix).** Replace $A(z)$ with a single linear map $A(z) = W_A\, z + b_A$. Then $u$ becomes linear in $z$ with $z$-independent coefficients:

$$u(x; z) \;=\; \underbrace{(\Phi(x)\, W_A)}_{V(x)}\, z \;+\; \Phi(x)\, b_A \;+\; b(x), \qquad \frac{\partial u}{\partial z} \;=\; V(x) \;\text{ (constant in } z).$$

Structurally this is a Compact-POD decoder: $V(x)$ is the reduced basis, $z$ is the coefficient vector, $b(x)$ is the mean field. The difference vs classical CP is that $V$ is learned end-to-end with the ViT encoder (rather than computed by SVD on snapshots) and $b$ is learned (rather than the training-set mean). Because the Jacobian is constant, the NM-ROM residual (nonlinear in $u$, but linear in $z$ through this decoder) reduces to one linear least-squares step per GN iter; in practice GN converges in 2 iters. Paired with `AffineNMROMSolver`, which precomputes $V(x_{\text{eq}})$ and $b(x_{\text{eq}})$ once per solve. Trained with a Laplacian residual loss $\lambda_{\text{lap}} \cdot \|K \cdot \text{decode}(z) - F\|^2$ evaluated at random interior 5-point stencils per batch; without it the encoder finds a good $z$ but the ROM-residual minimum is in a different direction (rel-L$^2$ 0.25); with it the encoder-vs-ROM $\|z\|$ agreement is within 1% and ROM rel-L$^2$ drops to ∼1e-2.

**Jacobian behavior at a glance.**

| **Decoder** | $\partial u/\partial z$ | **Per-iter Jacobian cost** | **Cold-start ROM** |
|---|---|---|---|
| ModulatedSIREN (jacfwd) | nonlinear in $z$ | latent_dim × $r$-eval | 2.88e-3 (with composite recipe) |
| ModulatedSIREN (vmap jacrev) | nonlinear in $z$ | $\sim 2 \times r$-eval | 2.88e-3 (3.2× faster Jac) |
| AffineZDecoder | nonlinear in $z$ | tiny MLP + jacrev | 0.2–0.8 (basin trap) |
| LinearAffineZDecoder | *constant* $V(x)$ | one precompute, then free | 9.81e-3 (with $\lambda_{\text{lap}}$) |